

```
1 import 'package:flutter/material.dart';
2
3 /// Enumerativi InfoType.
4 /// Servono a classificare la snackbar
5 /// e a far dipendere da quella scelta
6 /// (almeno) colore (e volendo anche icona,
7 /// testo, ecc.)
8 enum InfoType { success, error, info, warning }
9
10 /// classe InfoSnackBar.
11 /// Utilizzato per la visualizzazione di SnackBar
12 /// informative.
13 /// Permette di mostrare messaggi di feedback
14 /// all'utente specificando:
15 /// - il testo del messaggio
16 /// - la tipologia del messaggio (success, error, info, warning)
17 /// - la durata di visualizzazione
18 /// - un'eventuale azione associata (action label)
19 /// Passato il "type" come parametro
20 /// viene scelto di conseguenza il colore di
21 /// background.
22 /// La tipologia del messaggio determina automaticamente lo stile
23 /// della SnackBar (colore di sfondo e, se previsto, icona)
24 class InfoSnackBar {
25   /// metodo statico "show".
26   /// La classe "InfoSnackBar" è progettata come utility stateless;
27   /// per questo motivo il metodo "show" è statico
28   /// e può essere invocato senza istanziare la classe,
29   /// evitando la gestione di stato non necessario
30   static void show(
31     /// riferimento alla posizione di un widget
32     /// all'interno dell'albero dei widget di Flutter
33     BuildContext context, {
34       /// messaggio informativo
35       required String message,
36       /// tipologia di messaggio informativo
37       required InfoType type,
38       /// action label
39       String? actionLabel,
40       /// durata
41       required Duration duration,
42   }) {
43     /// Recuperare lo "ScaffoldMessenger" associato
44     /// allo schermo corrente
45     final messenger = ScaffoldMessenger.of(context);
46
47     /// Evitare "stack" di snackbar:
48     /// ne mostri una per volta.
49     /// Previene la sovrapposizione
50     /// o l'accodamento automatico,
51     /// garantendo una sola SnackBar
52     /// visibile per volta
53     messenger.hideCurrentSnackBar();
54
55     /// ritorna una SnackBar
56     messenger.showSnackBar(
57       SnackBar(
58         /// content (SnackBar)
59         content: Text(
60           /// messaggio/avviso
```

```
61     message,
62     // stile
63     style: TextStyle(
64       // colore del messaggio avviso
65       color: Colors.white,
66       // grassetto
67       fontWeight: FontWeight.bold,
68     ),
69   ),
70
71   /// action (SnackBar)
72   action: (actionLabel != null)
73     ? SnackBarAction(label: actionLabel, onPressed: () {})
74     : null,
75
76   /// backgroundColor
77   backgroundColor: _backgroundColor(type),
78
79   /// posizionamento SnackBar
80   behavior: SnackBarBehavior.floating,
81
82   /// durata SnackBar
83   /// decisa nel passaggio di parametri
84   duration: duration,
85
86   /// per definire la forma del bordo
87   shape:
88     /// tipo specifico di forma
89     /// per creare bordi arrotondati
90     RoundedRectangleBorder(
91       /// definisce il raggio di arrotondamento
92       /// di tutti e quattro gli angoli
93       borderRadius: BorderRadius.circular(15),
94     ),
95   ),
96 );
97 }
98
99 /// _backgroundColor
100 /// Restituisce il colore di sfondo della SnackBar
101 /// in base alla tipologia di messaggio ("InfoType")
102 static Color _backgroundColor(InfoType type) {
103   switch (type) {
104     /// successo (verde scuro/green)
105     case InfoType.success:
106       // return Colors.green
107       return const Color(0xFF2E7D32);
108
109     /// errore (rosso/red)
110     case InfoType.error:
111       // return Colors.red;
112       return const Color(0xFFC62828);
113
114     /// warning (arancione/orange)
115     case InfoType.warning:
116       // return Colors.orange;
117       return const Color(0xFFED6C02);
118
119     /// info (blu/blue)
120     case InfoType.info:
```

```
121         // return Colors.blue;
122         return const Color(0xFF1565C0);
123
124         /// default (grigio/grey)
125         default:
126         // return Colors.grey;
127         return const Color(0xFF9E9E9E);
128     }
129 }
130
131 /// _icon (metodo statico non utilizzato/mai richiamato)
132 /// Restituisce l'icona associata alla SnackBar
133 /// in base alla tipologia di messaggio ("InfoType")
134 static IconData _icon(InfoType type) {
135     switch (type) {
136         /// success
137         case InfoType.success:
138             return Icons.check_circle_outline;
139
140         /// error
141         case InfoType.error:
142             return Icons.error_outline;
143
144         /// warning
145         case InfoType.warning:
146             return Icons.warning_amber_rounded;
147
148         /// default (in tutti gli altri casi)
149         /// info
150         default:
151             return Icons.info_outline;
152     }
153 }
154 }
155
```